

Introduction to Python

Week 3

Program for Today: all about functions

1. How to define a function
2. How to call a function
3. Return statements
 1. Implicit return statements
 2. Multiple return statements
4. Global and local names
5. Arguments I
 1. Accepting positional arguments
 2. Undefined number of arguments
 3. Mutable arguments
6. Nested functions and Function Factories
7. Arguments II
 1. Accepting keyword arguments
 2. Positional and keyword arguments
 3. Defining defaults for missing arguments
8. Lambda functions in map, filter, reduce and in sorted
9. Programming notes

How to define a function

- A *function* is an object consisting of code that can be executed by calling the name of that function
- We define a function with a *function definition*
 - For example:

```
def adder(n1, n2):  
    total = n1 + n2  
    return total
```
 - The part of the definition starting with `def` and ending with the colon (`:`) is called the *function header*
 - The function header consists of the keyword `def`, the name of the function, and between brackets the parameters of the function
 - Everything below the function header with the same indentation is called the *function body*, and that is the code Python will execute when the function is called
 - The statement with the *return* keyword, returns a value to the caller of the function

How to call a function

- We call a function by writing round brackets after its name
 - Between these round brackets we can put arguments
 - For example, `a = adder(1, 2)`
- After a function is called by this function call, Python will first create objects with as values the values from the arguments and as names the names of the parameters in the function definition
- Next Python will execute the code in the function body
- When the arguments are mutable, Python doesn't create a new object, but attaches the name in the parameter list to the object in the argument list
- At the end of the execution, Python creates an object with as value the value returned by the function
- In our example, we add the name `a` to that object with the returned value, so we can use its value in our program by using the name `a`
- Forgetting to give the returned value a name or safeguard it in another way, e.g., in a file, is a common error

How to call a function

- Recall: we call a function by writing round brackets after its name
- A common error is using [], when actually you want to execute a function:
 - `print[1]` # Onscreen: `TypeError: 'function' object is not subscriptable`
- Another common error is using (), when you mean to take a slice []
 - `l1 = [1, 2, 3]`
`l1(1)` # Onscreen: `TypeError: 'list' object is not callable`
- These error messages are Python's slightly complicated way to tell you that you are using the wrong type of brackets

How to call a function

- The following code:

- ```
def adder(n1, n2):
 total = n1 + n2
 return total
a = adder(1, 2)
```

- Works the same as:

- ```
n1 = 1  
n2 = 2  
total = n1 + n2  
a = total
```

- Question: Why bother using functions?

- Using functions makes your code more readable
 - Reusing code is good for maintenance
 - You will make fewer errors if you write fewer lines of code
 - If you want to change something in the way you add, you only must make that change in one place
 - E.g. if `adder` is a function adding to bank accounts, maybe you want to add a logging function to `adder`

Return statement

- ```
def adder(n1, n2):
 total = n1 + n2
 return total
a = adder(1, 2)
```
- There are multiple things you can do with the result of a function:
  - You can use `return` in the function definition and give the result a name like we did in the above example
  - You can store the result in a database or another file type
  - You can print the result on the screen so a user can see it
- You must think about this, otherwise inadvertently the result will be gone when the function is finished
- A function leaves no traces, and you cannot use names that are defined inside a function, outside that function

# Return statement

- In the homework exercises where you define functions, we often say you must use `return`, so in that case you have no choice
  - Forgetting `return` or using `print` instead is a common mistake when doing homework
- Recall the example from the previous slide:
  - ```
def adder(n1, n2):  
    total = n1 + n2  
    return total  
a = adder(1, 2)
```
- An alternative, more concise style is:
 - ```
def adder(n1, n2):
 return n1 + n2
a = adder(1, 2)
```
  - Here you see that we do all the calculations in one step in the return statement. This is very common and seen as good practice.
  - Of course, here again we see the trade-off between concision and complexity



# Return statement

- Consider:

- ```
def calculate ():  
    result = 1  
    return result  
total = calculate()
```

- A return statement can only return one value

- Why then does the following work:

- ```
def calculate ():
 result1 = 1
 result2 = 2
 return result1, result2
total1, total2 = calculate() # total1 is now 1 and total2 is now 2
```

- Answer: last week we said the way to define a tuple is `t1 = (1, 2, 3)` and this is true, but an alternative way to define a tuple is `t1 = 1, 2, 3`

- So, what you saw in the code fragment above wasn't the return of two integer objects, but the return of exactly one tuple object

# Intermezzo 1: defining a tuple with one element and a nice trick

- `t1 = (1)` Does this work?
- Let's ask Python:
  - `print(type(t1)==tuple)` # Onscreen: False
  - `print(type(t1)==int)` # Onscreen: True
- Conclusion: the Python designers decided that `(1)` means the integer 1
  - The way to define a tuple with one element is:  
`t1 = 1, or`  
`t1 = (1,)`
- Tuples are the basis of a neat trick to swap values: `x, y = y, x`

# Implicit return statement

- ```
def adder(n1, n2):  
    total = n1 + n2  
a = adder(1, 2)
```
- If during execution of the function code Python gets to the end of the function, but didn't encounter a return statement, Python returns `None` for you
 - ```
def adder(n1, n2):
 if type(n1) == type(n2) == int:
 total = n1 + n2
 return total
a = adder('1', 2) # Returns: None
```
- You can test for `None`:
  - ```
if a != None:  
    print ('The result is: ' + a)  
else:  
    print ('Wrong numbers')
```
- You must know about this implicit return statement, as it is used a lot, but using this trick is bad: it makes your code more difficult to read

Multiple return statements

- ```
def calculator():
 n1 = int(input('First number? '))
 while (operator :=input('Operator?')) not in '+*':
 print("Only operators '+' and '*' allowed")
 n2 = int(input('Second number? '))
 if operator == '+':
 return n1+n2
 return n1*n2
```
- This works and in this simple case it is no big deal, but there is a trade-off between time to write this code and time to read this code

# Intermezzo 2: pass

- ```
def adder(n1, n2):  
    if type(n1) == int and type(n2) == int:  
        result = n1 + n2  
    else:  
        pass # What to do when inputs are no integers  
    return result  
a = adder(1, 2)  
print(a)
```
- `pass` doesn't do anything, so it can be used as a stub
- Question: why not do the following as a stub:
 - ```
else: # What to do when inputs are no integers
 return result
```
- Answer: this will give you an error, because after a colon you need an indented block

# Global and local names

- This is not a complete overview of scope, DataCamp (also here) does a very good job explaining. Here we will discuss some points in more detail
- Function call:
  - `print(adder(1, 2))`      # Onscreen: 3
- Function definition:
  - `def adder(n1, n2):`  
    `return n1 + n2`
  - When the function is called, python attaches the name `n1` to 1, and the name `n2` to 2
- The names `n1` and `n2` are *local* names that are only known inside the function, so the following gives you an error:
  - `def adder(n1, n2):`  
    `return n1 + n2`  
`adder(1,2)`  
`print(n1)`      # Onscreen: `NameError: name 'n1' is not defined`

# Global and local names

- A *global* name is defined outside any function and is known everywhere, even inside functions, except when the same name is also defined inside a function, then the local name dominates the global name
  - A global name can be defined via assignment:
    - `a = 1`
- A *local* name is defined when a function is called, and is only known inside that function
- A local name can be defined the same way as global names, but also in a function call
- ```
def func():  
    loc3 = 2
```
- When the function is called, Python first creates `loc3` as a local name. Next inside the function body, this name is assigned to an object with value 2
- ```
def func(loc1, loc2):
```
- When the following function is called, Python first creates `loc1` and `loc2` as a local name. Next these names will be assigned to values from the argument list

# Global and local names

- ```
def changer(n1):  
    n1 = 2  
    return n1  
n1 = 1  
changer(n1)  
print(n1) # Onscreen: 1
```
- The tricky part is that you may think you have only one name `n1`, but you have two names `n1`:
 - The local `n1`, that is only known inside the function
 - The global `n1`, that is known outside the function, but not anymore inside the function. As soon as the local name `n1` is defined, inside the function Python will take inside the function the local `n1`, when `n1` is used as a name
- When solving problems with scope in my head, I would name the objects: `local_n1` and `global_n1`

Global and local names

- If you define a name anywhere inside a function, that name will become a local name
 - Python makes a name local after the function is called and even before execution of the function body
- As expected, this works:
 - ```
def main (a):
 return a
b = 1
print(main (1)) # Onscreen: 1
```
- Question: Do you think, this works?
  - ```
def main (a):  
    b = b  
    return a  
b = 1  
print(main (1))  
  
# Onscreen: UnboundLocalError: local variable 'b' referenced before assignment
```
- Again, scopes can be very tricky. This problem is all about understanding timing

Global and local names

- Why do we get this error?
- When calling the function, the first thing Python, does is creating as local names, the names used in the parameter list and the names in the left side of assignment statements
- After that, the code in the function body be executed, and Python sees `b=b`
- Python now tries to create creates an object with the value `b`. To see what that value is Python first look at the local names. Python sees, there is a local name `b` but that hasn't yet a value
- Python throws an error

Global and local names

- The way to override the behavior where a local name dominates an equal global name, is by using the `global` keyword
 - If you use `global b`, when you use the name `b` in the function definition, it will no longer be seen as local, but as global
- For example:
 - ```
def main (a):
 global b
 b = b
 return a
b = 1
print(main (1)) # 1
```

# Global and local names

- It's better to avoid the use of `global`, even though it is used a lot, and you need to know how it works
- Instead of:
  - ```
def changer():  
    global n1  
    n1 = n1 + 1  
n1 = 1  
changer()  
print(n1) # Onscreen: 2
```
- Use:
 - ```
def changer(n1):
 n1 = n1 + 1
 return n1
a = 1
a = changer(a)
print(a) # Onscreen: 2
```
- So instead of using `global` we use the argument parameter mechanism for making data available to a function and `return` for making data available to the caller of a function

# Arguments

- *Arguments* are the values you transfer to a function when it is called
- Python assigns names from the parameter list of the function to these arguments
- The following slides give you an overview, what rules Python follows

# Accepting positional arguments

- Function call:

- `print (adder (1, 2))`      # Onscreen: 3

- Function definition:

- `def adder(n1, n2):`  
    `return n1 + n2`

- When the function is called, python attaches the local name `n1` to 1, and the local name `n2` to 2

# Accepting an undefined number of positional arguments

- Function call:

- `print(adder(1, 2, 3, 4, 5))`      # Onscreen: 15

- Function definition:

- ```
def adder(*args):  
    result = 0  
    for number in args:  
        result += number  
    return result
```

- When the function is called python attaches the name `args` to a list that contains all the arguments

- The use of `args` is a convention, Python is also ok if you use another name

Alternative way to deal with an undefined number of 'arguments'

- You could do the same with a self defined list
- For example:

- Function call:

- `print(adder([1, 2, 3, 4, 5]))` # Onscreen: 15

- Function definition:

- ```
def adder(l1):
 result = 0
 for number in l1:
 result += number
 return result
```

- The nice thing about using `*args` however, is not that it would be faster or something like that, but that just by seeing the function header, everybody knows that the function will accept an unlimited number of positional arguments



# Mix of accepting defined and undefined arguments

- You can mix defined and undefined arguments
- For example:
  - Function call:
    - `print(adder(1, 2, 3, 4, 5))`      # Onscreen: 15
  - Function definition:
    - ```
def adder(n1, n2, *args):  
    result = n1 + n2  
    for number in args:  
        result += number  
    return result
```
- Python first fills the defined parameters with arguments and after that fills the args list with the rest of the arguments

A problem with mutable arguments

- Function definition:

- ```
def adder(l2):
 result = 0
 while l2:
 result += l2.pop()
 return result
```

```
l1 = [1, 2, 3, 4, 5]
total = adder(l1)
print(f'The sum of {l1} = {total}')
```

- Question: what do you think the result is?

- It is: The sum of [] = 15

- Clearly the names `l1` and `l2` refer both to the same list object and as a list is mutable, disaster strikes

# A problem with mutable arguments

- You could solve this by creating a copy of the list at the start of the function:
  - ```
def adder(l2):  
    l2 = l2[:]  
    result = 0
```
- Note: this problem only exists with mutable arguments
- When we say that parameters are filled with arguments, we mean that:
 - When they are *mutable*, argument and parameter will refer to the same object
 - When they are *immutable*, the parameter will refer to a copy of the argument
- This is not a common problem, but when it happens it can be rather difficult to understand what is happening

Nested functions

- You can define a function inside another function, this is called a *nested function*
- The inner function is only visible to its outer function, but not outside it
- Functions are objects just like other objects and their names behave like any other name
 - For example:
 - ```
def main ():
 pass
print (type(main)) # Onscreen: <class 'function'>
```

# Function factories

- You can use the fact that functions behave like any other object to write a function that returns new functions based on a template function
- Our template function is:
  - ```
def adder(n1, n2):  
    return n1 + n2
```
- We now write a function called `function_factory`:
 - ```
def function_factory(increment):
 def adder(n1):
 return n1 + increment
 return adder
```
- With the help of the function factory, you can create two new specialist functions:
  - ```
add4 = function_factory(4)
```
 - ```
add5 = function_factory(5)
```
- And now that we gave names to our newly created functions, we can use them:
  - ```
print(add4(3))    # Onscreen: 7
```
 - ```
print(add5(3)) # Onscreen: 8
```

# Accepting keyword arguments

- Function call:

- `print(calc(fifth = 2, third=4, fourth=1, sixth=5, second=2, first=3))`  
# Onscreen: 63

- Function definition:

- `def calc(first, second, third, fourth, fifth, sixth):`  
    `return first + 2 * second + 3 * third + 4 * fourth \`  
        `+ 5 * fifth + 6 * sixth`

- When the function is called, python attaches the name `first` to the value 3, and the name `second` to the value 2 etc
- The nice thing about keyword arguments is that you don't have to think of the order of the parameters

# Accepting undefined keyword arguments

- Function call:

- `print(calc(third=4, fourth=1, second=2, first=3, fifth = 2, sixt=5))`  
# Onscreen: 63

- Function definition:

- `def calc(**kwargs):`  
    `return kwargs['first'] + 2 * kwargs['second']+ \`  
        `3 * kwargs['third'] + 4 * kwargs['fourth'] + \`  
        `5 * kwargs['fifth']+ 6 * kwargs['sixt']`

# You can mix positional and keyword arguments

- You have to follow the right order, because other orders lead to an error:
  - ```
def calc(first, second, *args, **kwargs):  
    return first + second * 2 + args[0] * 3 + args[1] * 4 +\  
        5 * kwargs['fifth'] + 6 * kwargs['sixth']
```
 - A call could for example be:
 - ```
print(calc(3, 2, 4, 1, sixth=5, fifth=2)) # Onscreen: 63
```



# The argument list of the call decides whether an argument is positional or keyword

- The function definition is:

```
def adder(x, y):
 print(f'x={x}, y={y}')
```

- The following function calls give all the same result ('x=1, y=2'):
- `adder(1, 2)`
- `adder(x = 1, 2)`
- `adder(1, y=2)`
- `adder(x=1, y=2)`
- So, the parameter list of the function definition doesn't tell you whether it is positional or keyword, that is decide by the call

# Defining defaults for missing arguments

- Keyword **parameters** are used to define defaults for missing arguments
- Function call:
  - `print(multiplier(3))`                      `# Onscreen: 6`
- Function definition:
  - `def multiplier(first, second=2):`  
    `return first * second`
- When the function is called, python attaches the name `first` to the value 3, and the name `second` to second argument and, as there is no second argument, to the default value of 2
- The names `second` and `first` are local names that are only known inside the function

# Defining defaults for missing arguments

- Using defaults looks a lot like keyword arguments, but remember:
  - Keyword arguments are part of the argument list in the function call,
  - Defaults are part of the parameter list in the function header
- You can combine them, what will be the result of
  - `print(multiplier(2, second=3))`
  - `def multiplier(first, second=2):`  
    `return first * second`
- `# Onscreen: 6`

# A problem with mutable defaults

- ```
def main (addition, l1 = []):  
    l1.append(addition)  
    return l1  
print(main(2))  
print(main(3))
```
- Question: what do you think the result is?
- It is:
[2]
[2, 3]
- The problem is that Python creates an object with the value for the default when it encounters the def statement and that l1 keeps referring to the same object all the time. While that object is changed by the code inside the function

A problem with mutable defaults

- You could solve this by never using mutable defaults

- ```
def main (addition, l1 = None):
 if l1 is None:
 l1 = []
 l1 += [addition]
 return l1
print(main(2))
print(main(3))
```

# Methods without arguments?

- A method is a function that 'belongs to' an object
- It often seems as if methods are called without arguments:
  - ```
l1 = [1,2,3]  
l1.sort()  
print(l1)
```
- But under the hood:
 - ```
l1 = [1,2,4]
list.sort(l1)
print(l1)
```
- So, the dot does do the same as putting the object as the first argument, between the round brackets
- When we discuss OO-design it will be clear to you what this means

# Lambda functions

- Consider the following lambda function:
  - `add_two_numbers = lambda x,y: x + y`  
`print(add_two_numbers(1,2))`
- You could do the same as follows:
  - `def add_two_numbers (x,y):`  
    `return x+y`  
`print(add_two_numbers(1,2))`
- So, what's the fun?

# Lambda functions

- Lambda functions are useful in combination with the map function:
  - `l1 = [1, 2, 3, 4, 5, 6]`  
`print(list(map(lambda x: x * 2, l1)))` # Onscreen: [2, 4, 6, 8, 10, 12]
- With the filter function:
  - `l1 = [1, 2, 3, 4, 5, 6]`  
`print(list(filter(lambda x: not x % 2, l1)))` # Onscreen: [2, 4, 6]
- Note: you need the list function to convert a map or filter into a list, otherwise you get:
  - `print(map(lambda x: x * 2, l1))`  
# Onscreen: <map object at 0x000001E5136D77F0>
- With the reduce function:
  - `from functools import reduce`  
`l1 = [1, 2, 3, 4, 5, 6]`  
`print(reduce(lambda x, y: x*y, l1))` # Onscreen: 720



# Lambda functions

- Lambda functions are useful in combination with the sort method and the sorted function:
  - Sorted function without using the key parameter:
    - `l1=['aaa', 'c', 'baab']`  
`print(sorted(l1))`  
# Onscreen: ['aaa', 'baab', 'c']
  - Sorted function with using the key parameter:
    - `print(sorted(l1, key = lambda x: x.count('a')))`  
# Onscreen: ['c', 'baab', 'aaa']
- Normal functions can also be used with the key parameter:
  - `print(sorted(l1, key = len))` # Onscreen: ['c', 'aaa', 'baab']
- The max function has also an optional key parameter, that works the same:
  - `d = {'a': 2, 'b': 1}`  
`print(max(d.items()))` # Onscreen: ('b', 1)
  - `print(max(d.items(), key = lambda x: x[1]))` # Onscreen: ('a', 2)

# Programming Notes: different type of loops

- You have a list with integers called `x`, If an integer `< 10` double it, `> 10` halve it, and if `== 10` you ignore it

## 1. For-loop:

```
result = []
for el in x:
 if el != 10:
 result.append(el * 2 if el < 10 else el // 2)
print(result)
```

## 2. While loop:

```
result = []
index = 0
while index < len(x):
 el = x[index]
 if el != 10:
 result += [el * 2 if el < 10 else el // 2]
 index += 1
print(result)
```

# Programming Notes: nested for-loops

- Say, you want to write a program that prints a list with all combinations of one of the elements of `x = ['a', 'b', 'c', 'd']`, one of the elements of `y = ['e', 'f']` and one of the elements of `z = ['g', 'h']`, and you want the printed result to look like:  
`['aeg', 'aeh', 'afg', 'afh', 'beg', etc.]`
- It seems a good idea to use for loops, as we have proper sequences to loop over
- What you must realize is that if you look at the result from right to left, the first letter in the output changes  $\text{len}(x) = 4$  times, the second letter changes  $\text{len}(x) * \text{len}(y) = 8$  times and the third letter changes  $\text{len}(x) * \text{len}(y) * \text{len}(z) = 16$  times

# Programming Notes: nested for-loops

- When writing the for loops, it is important that you start with the for loop of the list of which the element changes the least, and end with the for-loop of the list of which the element changes the most:
  - ```
result = []  
for elx in x:  
    for ely in y:  
        for elz in z:  
            result += [elx + ely + elz]  
print(result)
```
- So, remember the code inside the most inner loop is executed the most often

Question 3

- Assume that you already have a variable called 'x', which contains a dictionary. Modify 'x' with the dictionary `y = {'c': 3, 'd': 3}` in the following way: - add to 'x' all key-value pairs of 'y' for which the key is not also present in 'x' - delete from 'x' all key-value pairs for which the key is also present in 'y'
- `x = {'b': 2, 'c': 3}`
`y = {'c': 3, 'd': 3}`
- `for key, value in y.items():`
 `if key not in x:`
 `x[key] = value`
- `for key, value in y.items():`
 `if key in x:`
 `del x[key]`
- `print(x)`
- What will be printed?
- # Onscreen: `{'b': 2}`

Question 3

- Assume that you already have a variable called 'x', which contains a dictionary. Modify 'x' with the dictionary `y = {'c': 3, 'd': 3}` in the following way: - add to 'x' all key-value pairs of 'y' for which the key is not also present in 'x' - delete from 'x' all key-value pairs for which the key is also present in 'y'
- `x = {'b': 2, 'c': 3}`
`y = {'c': 3, 'd': 3}`
- ```
for key, value in y.items():
 if key not in x:
 x[key] = value
 del (y[key])
```
- ```
for key, value in y.items():  
    if key in x:  
        del x[key]
```
- `print(x)`
- What will be printed?
- # Onscreen: `RuntimeError: dictionary changed size during iteration`

Question 3

- Assume that you already have a variable called 'x', which contains a dictionary. Modify 'x' with the dictionary `y = {'c': 3, 'd': 3}` in the following way:
 - add to 'x' all key-value pairs of 'y' for which the key is not also present in 'x'
 - delete from 'x' all key-value pairs for which the key is also present in 'y'
- `x = {'b': 2, 'c': 3}`
`y = {'c': 3, 'd': 3}`
- ```
for key, value in y.items():
 if key not in x:
 x[key] = value
 else:
 del x[key]
```
- `print(x)`
- # Onscreen: `{'b': 2, 'd': 3}`

# Question 5

- Assume you have two lists: called 'keys' and 'values'. Same length. Both containing integers. The 'keys' list contains only unique numbers. Create a dictionary 'x', keys taken from 'keys', values from 'values', with the same index. Exceptions: values divisible by 5, the key and the value equal.
- ```
keys = [3, 2, 1, 4]
values = [3, 37, 60, 79]
```
- ```
x = {}
for key, value in zip(keys, values):
```
- ```
    if key == value:
        continue
    if value % 5 == 0:
        continue
```
- ```
 x[key] = value
print(x)
```
- # On screen: {2: 37, 4: 79}



# Question 5

- Assume you have two lists: called 'keys' and 'values'. Same length. Both containing integers. The 'keys' list contains only unique numbers. Create a dictionary 'x', keys taken from 'keys', values from 'values', with the same index.  
Exceptions: values divisible by 5, the key and the value equal.
- ```
keys = [3, 2, 1, 4]
values = [3, 37, 60, 79]
```
- ```
x = {}
for key, value in zip(keys, values):
```
- ```
    if key == value or value % 5 == 0:
        continue
```
- ```
 x[key] = value
print(x)
```
- # On screen: {2: 37, 4: 79}

# Question 5

- Assume you have two lists: called 'keys' and 'values'. Same length. Both containing integers. The 'keys' list contains only unique numbers. Create a dictionary 'x', keys taken from 'keys', values from 'values', with the same index.  
Exceptions: values divisible by 5, the key and the value equal.
- ```
keys = [3, 2, 1, 4]
values = [3, 37, 60, 79]
```
- ```
x = {}
for key, value in zip(keys, values):
```
- ```
    if not (key == value or value % 5 == 0):
        x[key] = value
print(x)
```
- # On screen: {2: 37, 4: 79}

Question 5

- Assume you have two lists: called 'keys' and 'values'. Same length. Both containing integers. The 'keys' list contains only unique numbers. Create a dictionary 'x', keys taken from 'keys', values from 'values', with the same index.
Exceptions: values divisible by 5, the key and the value equal.
- ```
keys = [3, 2, 1, 4]
values = [3, 37, 60, 79]
```
- ```
x = {}
for key, value in zip(keys, values):
```
- ```
 if key != value and value % 5 != 0):
 x[key] = value
print(x)
```
- # On screen: {2: 37, 4: 79}

# Question 9

- 'x' is a list containing integer values. 4 variables 'a', 'b', 'c', 'd', containing integers. Print a list containing strings that describe the elements of 'x' as follows:
- If greater than 'a', use "cat1", If less than 'b', use "cat2", If less than or equal to the value of 'c', use "cat3", If the element is greater than or equal to the value of 'd', use "cat4".
- If the element fits into more than one category, use the highest category.
- If the integer doesn't fall into any of the 4 categories, use "catunknown".

# Question 9

- `x = [5, 10, 15, 20]; a = 4; b = 9; c = 13; d = 15`  
`cats = []`  
`for element in x:`
  - `if element > a:`  
`cats.append("cat1")`  
`elif element < b:`  
`cats.append("cat2")`  
`elif element <= c:`  
`cats.append("cat3")`  
`elif element >= d:`  
`cats.append("cat4")`  
`else:`  
`cats.append("catunknown")`
- `print(cats)`
- `# Onscreen: ['cat1', 'cat1', 'cat1', 'cat1']`

# Question 9

- `x = [5, 10, 15, 20]; a = 4; b = 9; c = 13; d = 15`  
`cats = []`  
`for element in x:`
- - `if element >= d:`  
`cats.append("cat4")`
  - `elif element <= c:`  
`cats.append("cat3")`
  - `elif element < b:`  
`cats.append("cat2")`
  - `elif element > a:`  
`cats.append("cat1")`
  - `else:`  
`cats.append("catunknown")`
- `print(cats)`
- # Onscreen: `['cat3', 'cat3', 'cat4', 'cat4']`

# Question 9

- `x = [5, 10, 15, 20]; a = 4; b = 9; c = 13; d = 15`  
`cats = []`  
`for element in x:`
- `result = "catunknown"`  
`if element > a:`  
`result = "cat1"`  
`if element < b:`  
`result = "cat2"`  
`if element <= c:`  
`result = "cat3"`  
`if element >= d:`  
`result = "cat4"`  
`cats.append(result)`
- `print(cats)`
- # Onscreen: `['cat3', 'cat3', 'cat4', 'cat4']`